Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ЛАБОРАТОРНАЯ РАБОТА №7

По дисциплине Алгоритмы и структуры данных
Тема работы Жадные алгоритмы и динамическое программирование
Обучающийся Черненко Андрей Дмитриевич
Группа K4111c
Образовательная программа Разработка и эксплуатация промышленного бэкенда

Обучающийся _____ _____ Черненко А. Д.
(дата) (подпись) (Ф.И.О.)

Руководитель _____ _____ Мусаев А. А.
(дата) (подпись) (Ф.И.О.)

Санкт-Петербург, 2026

Содержание

ВВЕДЕНИЕ **2**

1 Задание 1. Жадный алгоритм для музейных экспонатов **2**

1.1 Условие 2

1.2 Алгоритм решения 2

1.3 Исходные данные и результат 3

2 Задание 2. Максимальная непрерывная возрастающая последовательность **4**

2.1 Условие 4

2.2 Алгоритм решения 4

2.3 Результат 5

3 Задание 3. Динамическое программирование для эвакуации **6**

3.1 Условие 6

3.2 Алгоритм решения 6

3.3 Результат 7

4 Задание 4. Динамическое программирование для раскроя **7**

4.1 Условие 7

4.2 Алгоритм решения 7

4.3 Результат 8

5 Сравнение алгоритмов **9**

ЗАКЛЮЧЕНИЕ **10**

ВВЕДЕНИЕ

Целью лабораторной работы является изучение жадных алгоритмов и повторение метода динамического программирования на задачах выбора объектов и раскроя материала.

В работе решаются следующие задачи:

1. выбор экспонатов с максимальной суммарной ценностью при ограниченном количестве заходов в музей;
2. поиск максимальной непрерывной возрастающей последовательности в массиве;
3. выбор предметов при ограничениях по весу и времени эвакуации;
4. выбор заказов для раскроя бумажного бруска с учетом стоимости резов.

Все программы реализованы на языке C#.

1 Задание 1. Жадный алгоритм для музейных экспонатов

1.1 Условие

Вор хочет вынести из музея экспонаты. Для каждого экспоната известны вес и цена. Вор может сделать M заходов, каждый раз унося не более K килограммов. Требуется определить набор экспонатов, который дает максимальную суммарную стоимость.

1.2 Алгоритм решения

Для выбора экспонатов используется жадная стратегия. Для каждого экспоната рассчитывается отношение цены к весу:

$$r_i = \frac{p_i}{w_i}.$$

Затем экспонаты сортируются по убыванию этого отношения. На каждом заходе выбирается очередной самый выгодный по отношению цена/вес экспонат, который помещается в оставшееся место рюкзака.

Такой подход быстро строит хороший вариант решения. Для задачи дробного рюкзака он является оптимальным, а для задачи с неделимыми экспонатами выступает жадной эвристикой и не всегда гарантирует глобальный оптимум.

```
static List<int> MuseumGreedy(
    (int weight, int price)[] items,
    int visits, int capacity)
{
    var order = items
        .Select((item, index) => new
        {
            Index = index + 1,
            item.weight,
            item.price,
            Ratio = (double)item.price / item.weight
        })
        .OrderByDescending(x => x.Ratio)
        .ToList();

    var result = new List<int>();
    var used = new bool[items.Length];

    for (int visit = 0; visit < visits; visit++)
    {
        int remaining = capacity;

        foreach (var item in order)
        {
            if (used[item.Index - 1] ||
                item.weight > remaining)
                continue;

            used[item.Index - 1] = true;
            result.Add(item.Index);
            remaining -= item.weight;
        }
    }

    return result;
}
```

Листинг 1: Жадный выбор экспонатов

1.3 Исходные данные и результат

Использованы параметры $M = 2$ и $K = 7$ кг.

Таблица 1: Экспонаты музея

№	Вес, кг	Цена, руб.	Цена/вес
1	4	10	2,50
2	7	16	2,29
3	3	8	2,67
4	5	14	2,80
5	2	4	2,00

Порядок выбора по отношению цена/вес: 4, 3, 1, 2, 5. На первом заходе выбираются экспонаты 4 и 5: общий вес 7 кг, стоимость 18 рублей. На втором заходе выбираются экспонаты 3 и 1: общий вес 7 кг, стоимость 18 рублей.

Итоговый набор: экспонаты 1, 3, 4 и 5. Общий вес равен 14 кг, а суммарная стоимость --- 36 рублей.

Временная сложность сортировки и выбора составляет $O(N \log N + MN)$, пространственная сложность --- $O(N)$.

2 Задание 2. Максимальная непрерывная возрастающая последовательность

2.1 Условие

Дан массив N, состоящий из случайных целых чисел в диапазоне от -100 до 100. Требуется найти наибольшую непрерывную возрастающую последовательность, для которой выполняется:

$$N[i] < N[i+1] < \ldots < N[i+m].$$

2.2 Алгоритм решения

Массив просматривается слева направо. Переменная `currentLength` хранит длину текущей возрастающей серии. Если текущий элемент больше предыдущего, длина серии увеличивается. Иначе серия начинается заново с текущего элемента. Одновременно сохраняется самая длинная найденная серия и ее начальный индекс.

```
static (int start, int length) LongestIncreasingRun(
```

```
    int[] values)
{
    if (values.Length == 0)
        return (-1, 0);

    int bestStart = 0;
    int bestLength = 1;
    int currentStart = 0;
    int currentLength = 1;

    for (int i = 1; i < values.Length; i++)
    {
        if (values[i] > values[i - 1])
        {
            currentLength++;
        }
        else
        {
            currentStart = i;
            currentLength = 1;
        }

        if (currentLength > bestLength)
        {
            bestStart = currentStart;
            bestLength = currentLength;
        }
    }

    return (bestStart, bestLength);
}
```

Листинг 2: Поиск максимальной возрастающей серии

2.3 Результат

Для массива

$$[-2,\ 1,\ 3,\ 2,\ 4,\ 5,\ 6,\ 0,\ 7,\ 8]$$

самая длинная непрерывная возрастающая серия имеет вид:

$$[2,\ 4,\ 5,\ 6].$$

Ее длина равна 4.

Алгоритм выполняет один проход по массиву, поэтому временная слож-

ность равна $O(n)$, а дополнительная память --- $O(1)$.

3 Задание 3. Динамическое программирование для эвакуации

3.1 Условие

В здании начался пожар. Для каждого предмета известны вес w_i, ценность v_i и время выноса t_i. Рюкзак выдерживает вес W, а общее время эвакуации ограничено значением T. Требуется получить максимальную суммарную ценность.

3.2 Алгоритм решения

Используется трехмерная таблица $dp[i][w][t]$, где i --- количество рассмотренных предметов, w --- использованный вес, t --- использованное время. Для каждого предмета сравниваются варианты «взять» и «не брать».

```
static int Evacuation(
    (int w, int v, int t)[] items,
    int maxWeight, int maxTime)
{
    int n = items.Length;
    int[,,] dp = new int[n + 1,
                         maxWeight + 1,
                         maxTime + 1];

    for (int i = 1; i <= n; i++)
    {
        var item = items[i - 1];

        for (int w = 0; w <= maxWeight; w++)
        for (int t = 0; t <= maxTime; t++)
        {
            dp[i, w, t] = dp[i - 1, w, t];

            if (item.w <= w && item.t <= t)
            {
                int candidate =
                    dp[i - 1, w - item.w, t - item.t]
                    + item.v;

                dp[i, w, t] =
```

```
                    Math.Max(dp[i, w, t], candidate);
            }
        }
    }

    return dp[n, maxWeight, maxTime];
}
```

Листинг 3: Эвакуация ценных предметов

3.3 Результат

Использованы предметы с параметрами:

$$(3, 6, 4),\ (4, 7, 5),\ (5, 10, 6),\ (2, 4, 3),\ (6, 12, 7),$$

где в каждой тройке указаны вес, ценность и время. Ограничения: $W = 10$ кг и $T = 12$ минут.

Оптимально выбрать предметы 2 и 5:

$$w = 4 + 6 = 10 \text{ кг}, \qquad t = 5 + 7 = 12 \text{ мин}, \qquad v = 7 + 12 = 19.$$

Максимальная ценность равна 19. Временная сложность алгоритма составляет $O(nWT)$, пространственная сложность --- $O(nWT)$.

4 Задание 4. Динамическое программирование для раскроя

4.1 Условие

Имеется бумажный брусок длины L и N заказов на куски. Каждый заказ характеризуется длиной d_i и ценой p_i. Каждый разрез приносит убыток C. Нужно выбрать заказы с максимальной прибылью:

$$P = \sum p_i - C \cdot k.$$

4.2 Алгоритм решения

Для каждого заказа вычисляется эффективная прибыль $p_i - C$. Далее решается задача о рюкзаке по одному ограничению: $dp[x]$ хранит максималь-

ную прибыль при занятой длине x. Перебор длины справа налево не позволяет использовать один заказ более одного раза.

```
static (int profit, List<int> selected) Cutting(
    int length, (int d, int p)[] orders, int cutCost)
{
    int[] dp = new int[length + 1];
    List<int>[] selected = new List<int>[length + 1];

    for (int i = 0; i <= length; i++)
        selected[i] = new List<int>();

    for (int i = 0; i < orders.Length; i++)
    {
        int size = orders[i].d;
        int gain = orders[i].p - cutCost;

        for (int used = length; used >= size; used--)
        {
            int candidate = dp[used - size] + gain;

            if (candidate > dp[used])
            {
                dp[used] = candidate;
                selected[used] =
                    new List<int>(selected[used - size]);
                selected[used].Add(i + 1);
            }
        }
    }

    return (dp[length], selected[length]);
}
```

Листинг 4: Раскрой бумажного бруска

4.3 Результат

Пусть $L = 20$ см, $C = 2$ рубля, а заказы имеют параметры:

$$(6, 11),\ (8, 14),\ (5, 9),\ (7, 13),\ (4, 6).$$

Оптимально выбрать заказы 2, 3 и 4:

$$d = 8 + 5 + 7 = 20 \text{ см},$$

$$\sum p_i = 14 + 9 + 13 = 36 \text{ руб.}, \qquad C \cdot k = 2 \cdot 3 = 6 \text{ руб.}$$

Итоговая максимальная прибыль:

$$P = 36 - 6 = 30 \text{ руб.}$$

Временная сложность алгоритма равна $O(NL)$, пространственная сложность --- $O(L)$.

5 Сравнение алгоритмов

Таблица 2: Сравнение алгоритмов лабораторной работы

Задача	Основная структура	Время	Память
Жадный выбор	Отсортированный список	$O(N \log N + MN)$	$O(N)$
Возраст. серия	Массив и счетчики	$O(n)$	$O(1)$
Эвакуация	Трехмерная таблица	$O(nWT)$	$O(nWT)$
Раскрой	Таблица по длине	$O(NL)$	$O(L)$

Жадный алгоритм принимает локально выгодное решение на каждом шаге и работает быстро, но для неделимых предметов может не дать глобальный оптимум. Динамическое программирование рассматривает больше состояний, зато позволяет получить точный результат при целочисленных ограничениях. Линейный алгоритм поиска возрастающей серии является наиболее простым по времени и памяти.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы реализован жадный алгоритм выбора музейных экспонатов. Для тестового набора он выбрал четыре экспоната общей стоимостью 36 рублей при двух заходах по 7 кг.

Также реализован линейный поиск максимальной непрерывной возрастающей последовательности. Для заданного массива найдена серия $[2, 4, 5, 6]$ длиной 4.

Дополнительно повторно применен метод динамического программирования: для задачи эвакуации получена максимальная ценность 19, а для задачи раскроя --- максимальная прибыль 30 рублей. Проведенное сравнение показало, что жадные алгоритмы проще и быстрее в реализации, но динамическое программирование надежнее для задач с несколькими ограничениями и требованием точного оптимума.